ClaimGuard Benchmarking

A simple explanation of the calculations, graphs, approvals and evidence

1. What do P90, minimum and maximum mean?

The system first maps differently written descriptions to one standard repair item. For example, **Oil Disposal**, **Waste Oil Disposal** and **Oil/Filter Disposal** are grouped as **Oil & Filter Disposal**.

Value	Simple meaning
Minimum	Lowest earlier price
Maximum	Highest earlier price
Average	Total of the prices divided by the count
Median	Middle price after sorting
Mode	One price that repeats most often; blank when there is no clear mode
Count	Number of earlier observations used
P90	A price that about 90% of earlier observations are at or below

Example historical prices: £4.00, £4.50, £4.50, £5.00, £5.50, £6.00, £6.50, £7.00. Using the Excel-style interpolated calculation, P90 = £6.65.

How is the percentage calculated?

If the new invoice price is £7.50: **difference = £7.50 - £6.65 = £0.85**. Percentage above P90 = £0.85 / £6.65 x 100 = **12.8%**.

A real challenge is raised only when the price is above the selected 5% or 10% threshold and the difference is at least £5. The current invoice is never included in its own benchmark, and at least three earlier matching prices are required.

2. What does the Knowledge Graph show?

The graph connects a **repairer** to a **standard repair item** when that repairer has invoice lines above the P90 challenge threshold. It shows the number of affected invoices, challenged lines, total difference, largest difference and exact supporting invoices.

Its purpose is to answer: Which repairers repeatedly charge above benchmark for which repair items?

Approvals, missing values and price evidence

What the application does now

3. Why did Approve fail and why did rows show Missing?

Approve previously failed because an uploaded invoice refreshed before the ontology comparison ran. The row therefore had no repair-item ID to approve. This has been fixed. The flow is now:

Upload -> Extract -> Map -> Compare prices -> Refresh the screen

Mappings above 95% confidence are accepted automatically. Manual price approval is available only after the line has a valid repair-item mapping and a positive challenge.

Missing can also be a correct status. It normally means there are fewer than three earlier matching invoices, no valid P90 yet, or the line does not have comparable price or unit information. In the final ten-invoice test, all **154 of 154** extracted lines mapped successfully.

4. Does the tool use only P90?

No. The final supported price considers three separate evidence sources:

Evidence	How it is used
Uploaded-invoice P90	Market ceiling calculated from earlier matching uploaded invoices
Approved ontology price	Standard governed price for the mapped repair item
Historical claims	Eligible earlier claim prices, using a recency-weighted median

When ontology and history are both reliable, the governed price is **60% ontology + 40% historical weighted median**. The final support uses the higher reliable ceiling from P90 and the governed price, but never more than the amount billed.

The LLM only helps select a repair category from the approved ontology. It never invents or supplies a price. All prices come from stored evidence.

5. Why did Review Matches show No Match?

Some descriptions were originally given the wrong type. For example, **Fit Track Rod Ends** was treated as a part instead of labour, and **Waste Oil and Filter** was treated as a part instead of disposal. These cases and the automatic post-upload comparison have been fixed.

A genuinely unknown repair description can still show No Match. That is intentional: a reviewer must select or approve a real ontology item instead of allowing the system to invent one.

Sharing through Git

Yes, it is easy to share. Push the project source to a private Git repository and send her access. Do not commit **.env**, API keys, local databases, **node_modules**, Python virtual environments or generated build folders. She can clone the repository, copy **.env.example** to **.env**, add her approved keys and follow the README setup commands.

Final verified result: 10 invoices, 154 extracted lines, 154 mapped, 0 unmatched.